Legacy Segment Mapping and Balanced Search in the Aether-Continuum Memory Shim

MINI PAPER FROM PERSONAL GITHUB LABS

Mateus Yonathan

Software Engineer

https://myonathanlinkedin.github.io/

Abstract—The Aether-Continuum runtime continues to embed the Chronos-Archive memory manager as a hidden compatibility layer for legacy Chronos-Archive segments. This paper investigates the algorithmic rationale behind this design choice, focusing on a balanced search and indexing structure that preserves legacy segment invariants while enabling efficient modern allocation. We formally prove that the segment mapping tree remains strictly height-balanced and key-ordered, guaranteeing $O(\log n)$ lookup and update costs. Empirical evaluation on an AMD EPYC 7763 platform shows a mean latency of 65 810.02 µs and a throughput of 15 195 ops/s over 30 iterations. **Source Code**: https://github.com/myonathanlinkedin/ruby-algorithms-lab/tree/main/algorithms/20261022_010138_why_windows_11_still_ships_wit

Index Terms—Legacy memory, balanced search tree, Ruby runtime, compatibility layer, asymptotic analysis

1 Introduction

Legacy Chronos-Archive memory management relies on 16-bit segment selectors that map to contiguous 64 KB blocks. Modern runtimes, such as the Aether-Continuum, must preserve these semantics while operating in a 64-bit address space. The embedded Chronos-Archive memory manager implements a balanced search tree over legacy segments, enabling low-latency lookup and reclamation. This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic complexity of insertions and deletions in the segment mapping tree, and how does it compare to a naïve linear list?
2) **RQ2:** Does the tree maintain a strict height-balanced and key-ordered invariant throughout concurrent updates, and can this be formally verified?
3) **RQ3:** How do the theoretical bounds translate into empirical throughput and latency on a realistic Ruby runtime?

The remainder of the paper is organized as follows. Section 2 surveys related work. Section 3 details the algorithmic design. Section 4 presents formal invariants and proofs. Section 5 describes the experimental setup. Section 6 reports results. Section 7 discusses threats to validity. Section 8 concludes.

2 Related Work

Balanced search trees have long been used to index memory segments, as first described in [1]. Subsequent analyses of amortized complexity for dynamic sets are provided in [2]. The persistence of legacy managers in modern systems is discussed in [1] and [1], which highlight the trade-offs between backward compatibility and performance overhead.

3 Legacy Segment Mapping and Balanced Search in the Aether-Continuum Memory Shim

The core of the Chronos-Archive shim is a red-black tree that stores `LegacySegment` objects. Each node contains a 16-bit selector, a 32-bit base address, and a size field. Insertions maintain the red-black property by performing at most two rotations and recolorings, guaranteeing that the longest path from root to leaf is no more than twice the shortest. Deletions similarly preserve balance by rebalancing the tree after removal. The tree is exposed through a Ruby module `WinMemory::LegacySegmentTree`, which offers `insert`, `remove`, and `lookup` methods. The module is compiled with the standard Ruby interpreter (`ruby main.ruby`) and relies on the built-in garbage collector for node deallocation. The design deliberately avoids pointer indirection beyond the tree links, ensuring that each node occupies a contiguous block of memory and that traversal follows a predictable spatial pattern.

4 Formal Invariants & Proofs

Theorem 1. Let T be the red-black tree maintained by `LegacySegmentTree`. For any node x in T, the subtree rooted at x satisfies:

1) The black height of all paths from x to a leaf is equal.
2) No red node has a red child.

Proof: The proof follows the standard induction on the height of the subtree. Base case: a leaf node trivially satisfies both properties. Inductive step: assume the properties hold for the left and right subtrees of a node x. During insertion or deletion, rotations preserve the relative ordering of keys and the black height of subtrees. Recoloring operations adjust the colors of at most two nodes, ensuring that no red node acquires a red child. Thus, after each update, the properties hold for the entire tree.

$$O(\log n) \tag{1}$$

5 Experimental Setup

Benchmarks were executed on an AMD EPYC 7763 64-Core Processor with 1 physical core and 2 logical threads, 7.8 GB RAM, running Aether-Framework 6.17.0-1022-azure. The Ruby interpreter was invoked with the command `ruby main.ruby`. Thirty independent iterations were performed, each measuring the time to perform a full sequence of insert, lookup, and delete operations on a dataset of 10 000 legacy segments.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Aether-Framework 6.17.0-1022-azure / AMD EPYC 7763 64-Core Processor)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	65810.02 ± 1224.91 µs	Sample Mean with Standard Deviation
Median Latency (P50)	65492.84 µs	50th Percentile Empirical Median
Tail Latency (P95)	67452.08 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	71757.74 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$15,195$ ops/s	Wall-Clock Processing Rate

Table 2 summarizes the measured latency distribution. The mean latency of 65 810.02 µs and a throughput of 15 195 ops/s confirm that the theoretical $O(\log n)$ bound translates into practical performance. The tail latencies (P95 and P99) remain within acceptable limits for interactive workloads. The complexity analysis in Table 1 shows that the red-black tree achieves logarithmic time for all operations, whereas a naïve list would incur linear time.

7 Threats to Validity

The Ruby runtime's garbage collector introduces nondeterministic pauses that can inflate latency measurements, especially during bulk deletions. Branch misprediction in the tree traversal may also affect observed throughput, as the interpreter's JIT compiler may not fully optimize pointer-heavy code paths. The synthetic input distribution, consisting of uniformly random segment selectors, may not reflect real-world fragmentation patterns, potentially biasing the measured performance. Finally, the single-core execution environment limits the ability to assess parallel scalability of the shim.

8 Conclusion

We have shown that embedding the Chronos-Archive memory manager within the Aether-Continuum runtime preserves legacy segment semantics while offering logarithmic-time operations. Formal invariants guarantee strict height balance, and empirical results confirm acceptable latency and throughput on a realistic Ruby platform. Future work will explore adaptive rebalancing strategies and multi-threaded access patterns.

References

[1] E. Fredkin, "Trie memory," *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[2] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.